

型，使得两者能够在训练过程中相互适应，从而提高整体系统的性能。尽管白盒增强架构可以有效改善 RAG 的性能，但也存在明显缺点。这种架构通常需要大量计算资源和时间来训练，特别是协同微调策略，需要大量的运算资源来实现语言模型和检索器的同步更新。

6.3 知识检索

在 RAG 中，检索的效果（召回率、精度、多样性等）会直接影响大语言模型的生成质量。以“树袋熊一般在哪里生活？”这个问题为例。如果检索器返回的外部知识是关于“树袋熊”名称相近的动物“袋熊”的相关知识，那么这些不正确的外部知识可能引导大语言模型生成错误答案。此外，检索的时间也是 RAG 总耗时的关键部分，因此检索的效率将影响用户的使用体验。优化检索过程，提升检索的效果和效率，对改善 RAG 的性能具有重要意义。针对优化检索过程，本节系统的对知识库构建、查询增强、检索器、检索结果重排序等关键技术进行梳理和介绍。

6.3.1 知识库构建

知识库构成了 RAG 系统的根基。正如古语所言：“巧妇难为无米之炊”，只有构建了全面、优质、高效的知识库，检索才能有的放矢，检索效果才能有保障。在 RAG 框架中，知识库构建主要涉及**数据采集及预处理**与**知识库增强**两个步骤。本小节将对这两个步骤分别展开介绍。

1. 数据采集及预处理

数据采集与预处理为构建知识库提供“原材料”。在构建文本型知识库的**数据采集**过程中，来自不同渠道的数据被整合、转换为统一的文档对象。这些文档对象不仅包含原始的文本信息，还携带有关文档的元信息（Metadata）。元信息可以用

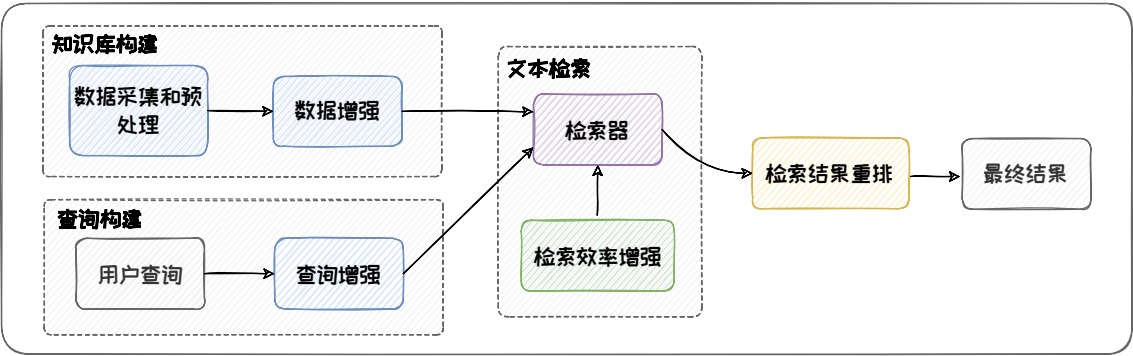


图 6.14: 知识检索流程图。

于后续的检索和过滤。以维基百科语料库的构建为例，数据采集主要通过提取维基百科网站页面内容来实现。这些内容不仅包括正文描述的内容，还包括一系列的元信息，例如文章标题，分类信息，时间信息，关键词等。

在采集到相应的数据后，还需通过**数据预处理**来提升数据质量和可用性。在构建文本型知识库时，数据预处理主要包括数据清洗和文本分块两个过程。**数据清洗**旨在清除文本中的干扰元素，如特殊字符、异常编码和无用的 HTML 标签，以及删除重复或高度相似的冗余文档，从而提高数据的清晰度和可用性。**文本分块**是将长文本分割成较小文本块的过程，例如把一篇长文章分为多个短段落。对长文本进行分块有两个好处：一是为了适应检索模型的上下文窗口长度限制，避免超出其处理能力；二是通过分块可以减少长文本中的不相关内容，降低噪音，从而提高检索的效率和准确性。

文本分块的效果直接影响后续检索结果的质量 [14]。如果分块处理不当，可能会破坏内容的连贯性。因此，制定合适的分块策略至关重要，包括确定切分方法（如按句子或段落切分）、设定块大小，以及是否允许块之间有重叠。文本分块的具体实施流程通常开始于将长文本拆解为较小的语义单元，如句子或段落。随后，这些单元被逐步组合成更大的块，直到达到预设的块大小，构建出独立的文本片段。为了保持语义连贯性，通常还会在相邻的文本片段之间设置一定的重叠区域。

2. 知识库增强

知识库增强是通过改进和丰富知识库的内容和结构，以提升其质量和实用性。这一过程通常涉及**查询生成与标题生成** [56] 等多个步骤，以此为文档建立语义“锚点”，方便检索时准确定位到相应文本。

查询生成指的是利用大语言模型**生成与文档内容紧密相关的伪查询**。这些伪查询从查询的角度来表达文档的语义，可以作为相关文档的“键”，供检索时与用户查询进行匹配。通过这种方式，可以增强文档与用户查询的匹配度。例如，对于一篇介绍考拉和树袋熊关系的文档，生成的查询“考拉和树袋熊之间的关系是什么？”不仅准确反映了文档的主题，还能有效引导检索器更精确的检索到与用户提问相关的信息。

标题生成指的是利用大语言模型**为没有标题的文档生成合适的标题**。这些生成的标题提供了文档的关键词和上下文信息，能来用来帮助快速理解文档内容，并在检索时更准确地定位到与用户提问相关的信息。对于那些原始文档中缺乏标题的情况，通过语言模型生成标题显得尤为重要。

6.3.2 查询增强

知识库涵盖的知识表达形式是有限的，但用户的提问方式却是千人千面的。用户遣词造句的方式以及描述问题的角度可能会与知识库中的存储的文本间存在差异，这可能导致用户查询和知识库之间不能很好匹配，从而降低检索效果。为了解决此问题，我们可以对用户查询的语义和内容进行扩展，即查询增强，以更好的匹配知识库中的文本。本小节将从**查询语义增强**和**查询内容增强**两个角度出发，对查询增强技术进行简要介绍。

1. 查询语义增强

查询语义增强旨在通过**同义改写**和**多视角分解**等方法来扩展、丰富用户查询

的语义，以提高检索的准确性和全面性。接下来分别对同义改写和多视角分解进行简要介绍。

(1) 同义改写

同义改写通过将原始查询改写成相同语义下不同的表达方式，来解决用户查询单一的表达形式可能无法全面覆盖到知识库中多样化表达的知识。改写工作可以调用大语言模型完成。比如，对于这样一个原始查询：“考拉的饮食习惯是什么？”，可以改写成下面几种同义表达：1、“考拉主要吃什么？”；2、“考拉的食物有哪些？”；3、“考拉的饮食结构是怎样的？”。每个改写后的查询都可独立用于检索相关文档，随后从这些不同查询中检索到的文档集合进行合并和去重处理，从而形成一个更大的相关文档集合。

(2) 多视角分解

多视角分解采用分而治之的方法来处理复杂查询，将复杂查询分解为来自不同视角的子查询，以检索到查询相关的不同角度的信息。例如，对于这样一个问题：“考拉面临哪些威胁？”，可以从多个视角分解为：1、“考拉的栖息地丧失对其有何影响？”；2、“气候变化如何影响考拉的生存？”；3、“人类活动对考拉种群有哪些威胁？”；4、“自然灾害对考拉的影响有哪些？”等子问题。每个子问题能检索到不同的相关文档，这些文档分别提供来自不同视角的信息。通过综合这些信息，语言模型能够生成一个更加全面和深入的最终答案。

2. 查询内容增强

查询内容增强旨在通过生成与原始查询相关的背景信息和上下文，从而丰富查询内容，提高检索的准确性和全面性 [58]。与传统的仅依赖于检索的方式相比，查询内容增强方法通过引入大语言模型生成的辅助文档，为原始查询提供更多维度的信息支持。

生成背景文档是一种查询内容增强的方法。它指的是在原始查询的基础上，利

用大语言模型生成与查询内容相关的背景文档。例如，对于用户查询“如何保护考拉的栖息地?”，可以生成以下背景文档：

考拉是原产于澳大利亚的树栖有袋类动物，主要分布在东部和东南部沿海的桉树林中。这些地区提供了考拉主要食物来源——桉树叶。考拉的栖息地包括开阔的森林和木林地，这里桉树丰富，不仅提供食物，还提供栖息和保护。考拉高度依赖特定种类的桉树，它们的分布与这些树木的可用性密切相关。

这些生成的背景文档可以作为原始查询的补充信息，提供更多的上下文内容，从而提高检索结果的相关性和丰富性。

6.3.3 检索器

给定知识库和用户查询，检索器旨在找到知识库中与用户查询相关的知识文本。检索器可分为**判别式检索器**和**生成式检索器**两类。本小节将对这两类检索器分别展开介绍。

1. 判别式检索器

判别式检索器通过判别模型对查询和文档是否相关进行打分。判别式检索器通常分为两大类：**稀疏检索器**和**稠密检索器**。稀疏检索器利用离散的、基于词频的文档编码向量进行检索，而稠密检索器则利用神经网络生成的连续的、稠密向量对文档进行检索。下面将详细的介绍这两种检索器以及代表性方法。

(1) 稀疏检索器

稀疏检索器（Sparse Retriever）是指使用**稀疏表示方法**来匹配文本的模型。这类检索器通过统计文档中特定词项出现的统计特征来对文档进行编码，然后基于此编码计算查询与知识库中的文档的相似度来进行检索。典型的稀疏检索技术包

括 TF-IDF[2] 和 BM25[43] 等，它们通过分析词项的分布和频率来评估文档与查询的相关性。TF-IDF 基于词频 (TF) 和逆文档频率 (IDF) 来衡量词语在文档或语料库中的重要性，然后用此重要性对文本进行编码。词频 (TF) 表示词语在文档中的出现频率，计算公式为：

$$\text{tf}_{i,j} = \frac{n_{i,j}}{\sum_k n_{k,j}}, \quad (6.1)$$

其中， $n_{i,j}$ 是词语 t_i 在文档 d_j 中的出现次数， $\sum_k n_{k,j}$ 是文档 d_j 中所有词语的出现次数之和，用于进行标准化以避免偏向长文档。逆文档频率 (IDF) 衡量词语的普遍性，计算公式为：

$$\text{idf}_i = \log \frac{|D|}{|\{j : t_i \in d_j\}|}, \quad (6.2)$$

其中， $|D|$ 是总文档数， $|\{j : t_i \in d_j\}|$ 是包含词语 t_i 的文档数。最终，TF-IDF 值为：

$$\text{tfidf}_{i,j} = \text{tf}_{i,j} \times \text{idf}_i. \quad (6.3)$$

TF-IDF 通过高词频和低文档频率产生高权重，倾向于过滤常见词语，保留重要词语。

BM25 是一种改进的文本检索算法，它在 TF-IDF 基础上通过文档长度归一化和词项饱和度调整，更精确地评估词项重要性，优化了词频和逆文档频率的计算，并考虑了文档长度对评分的影响。虽然不涉及词项上下文，但是 BM25 在处理大规模数据时表现优异，广泛应用于搜索引擎和信息检索系统。

(2) 稠密检索器

稠密检索器一般利用 **预训练语言模型** 对文本生成 **低维、密集** 的向量表示，通过计算向量间的相似度进行检索。按照所使用的模型结构的不同，稠密检索器大致可以分为两类：**交叉编码类** (Cross-Encoder)、**双编码器类** (Bi-Encoder)。二者结构分别如图 6.15 (a) 和 6.15 (b) 所示。

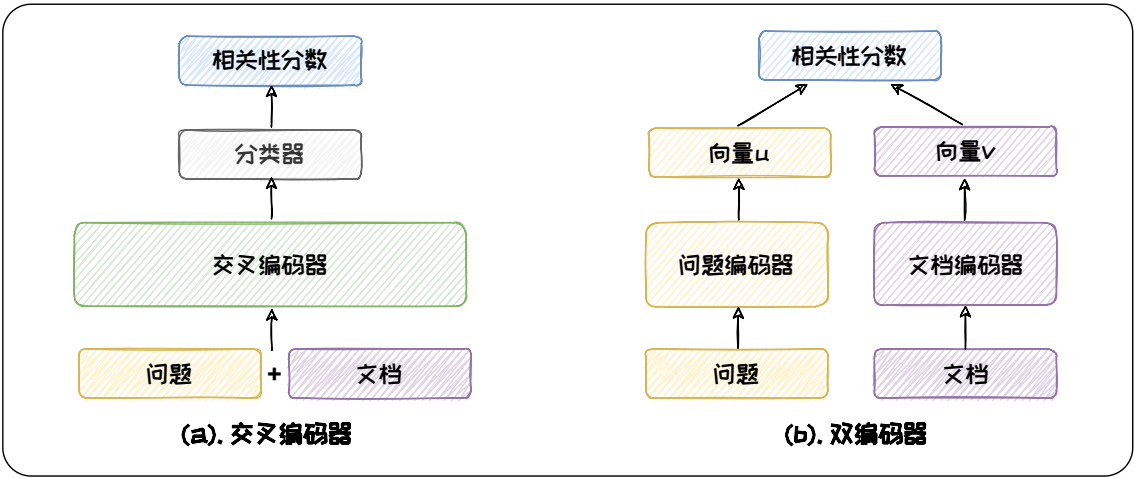


图 6.15: 不同稠密检索器对比图。

交叉编码类

交叉编码类“端到端”的给出查询和文档的相似度。这类模型将查询和文档拼接在一起，随后利用预训练语言模型作为编码器（例如 BERT）生成一个向量表示。接着，通过一个分类器处理这个向量，最终输出一个介于 0 和 1 之间的数值，表示输入的查询和文档之间的相似程度。其优点在于模型结构简单，能够实现查询和文档之间的深度交互，例如，在工作 [12, 44] 中，研究者们使用了交叉编码器来提升检索性能。然而，由于交叉编码类模型需要进行高复杂度的交叉注意力操作，计算量大，因此不适合在大规模检索阶段使用。这种模型更适用于对少量候选文档进行更精确排序的阶段，可以显著提升检索结果的相关性。

双编码器类

与交叉编码类模型不同，双编码类模型采用了一种“两步走”的策略。第一步，查询和文档首先各自通过独立的编码器生成各自的向量表示；第二步，对这两个向量之间的相似度进行计算，以评估它们的相关性。这种方法的优势在于，它允许预先离线计算并存储所有文档的向量表示，在线检索时则可直接进行向量匹配。因此，双编码器非常适合在工业环境中部署，具有极高的匹配效率。然而，在这种分离的处理方式中，查询与文档在提取特征向量时缺乏交互。这可能会对匹配的精度

确度产生影响。DPR (Dense Passage Retriever) [23] 是稠密检索器的一个代表工作。其使用两个独立的 BERT 编码器，分别将查询和文档映射到低维特征向量，然后通过向量点积衡量相似度。为了缓解查询与文档缺乏交互的问题，DPR 通过对比学习优化编码器，最大化查询与相关段落相似度的同时最小化与负面段落相似度。

为了缓解查询与文档在提取特征向量时缺乏交互的问题，可以在双编码器的基础上引入查询与文档的交互，以进一步提升双编码器的效果。ColBERT[24] 是其中的代表性方法。其以查询和文档间的 Token 级的相似度为度量，然后通过对比学习对双编码器进行微调，以使双编码器编码的特征向量可以兼顾查询和文档。此外，在 RAG 中，我们有时会对查询加入大段上下文进行增强，如第 3.3.2 节所介绍的情形。此时，查询的长度急剧增长，传统的检索方法可能难以有效的处理这些长查询。为解决此问题，可以采用 Poly-encoder[16]。其模型架构沿用双编码器的形式，但是它使用 m 个向量来捕获长查询的多个特征，而不是像普通双编码器那样只用一个向量来表示整个查询。并且，这 m 个向量随后与文档的向量通过注意力机制进行交互，其中的注意力模块采用查询和文档间的对比学习进行训练。

2. 生成式检索器

生成式检索器通过生成模型对输入查询直接生成相关文档的标识符 [29]。与判别式检索器不断地从知识库中去匹配相关文档不同，生成式检索器直接将知识库中的文档信息记忆在模型参数中。然后，在接收到查询请求时，能够直接生成相关文档的标识符（即 DocID），以完成检索 [48]。生成式检索器通常采用基于 Encoder-Decoder 架构的生成模型，如 T5[41]、BART[27] 等。生成式检索器的训练过程通常分为两个阶段 [29]。在第一阶段，模型通过序列到序列的学习方法，学习如何将查询映射到相关的文档标识符。这一阶段主要通过最大似然估计 (MLE) 来优化模型，确保生成的文档标识符尽可能准确。在第二阶段，通过数据增强和排名优化进一步提高检索效率和准确性。数据增强主要通过生成伪查询 [51] 或使用文

档片段 [61] 作为查询输入，以增加训练数据的多样性和覆盖面。排名优化则涉及使用特定的损失函数，如对比损失或排名损失，来调整模型生成文档标识符的顺序和相关性，从而更好地匹配查询的需求。

在生成式检索器中，DocID 的设计至关重要。其需要在语义信息的丰富性与标识符的简洁性之间取得平衡。常用的 DocID 形式分为两类：基于数字的 DocID 和基于词的 DocID。基于数字的 DocID 方法使用唯一的数字值或整数字符串来表示文档，虽然构建简单，但在处理大量文档时可能导致标识符数量激增，增加计算和存储负担。相比之下，基于词的 DocID 方法直接从文档的标题、URL 或 N-gram 中提取表示 [9]，能更自然地传达文档的语义信息。通常，标题是最佳选择，因为它提供了文档的宏观概述。但在缺乏高质量标题时，URL 或 N-gram 也可作为有效的替代方案。

尽管生成式检索器在性能上取得了一定的进步，但与稠密检索器相比，其效果仍稍逊一筹。此外，生成式检索器还面临着一系列挑战，包括如何突破模型输入长度的限制、如何有效处理大规模文档以及动态新增文档的表示学习等，这些都是亟待解决的问题。

6.3.4 检索效率增强

知识库中通常包含海量的文本，对知识库中文本进行逐一检索缓慢而低效。为提升检索效率，可以引入向量数据库来实现检索中的高效向量存储和查询 [39]。向量数据库的核心是设计高效的相似度索引算法。本节将简要介绍常用的相似度索引算法，以及用于构建向量数据库的常见软件库。

1. 相似度索引算法

在向量检索中，常用的索引技术主要分成三大类：基于空间划分的方法、基于量化方法和基于图的方法。

基于空间划分的方法将搜索空间划分为多个区域来实现索引，主要包括基于树的索引方法和基于哈希的方法两类。其中，基于树的索引方法通过一系列规则递归地划分空间，形成一种树状结构，每个叶节点代表一个较小区域，区域内的数据点彼此接近。在查询时，算法从树的根节点出发，逐步深入到合适的叶节点，最后在叶节点内部进行数据点的相似度比较，以找到最近的向量。常见的基于树的索引包括 KD 树 [4] 和 Ball 树 [13] 等。而基于哈希的方法（如局部敏感哈希（LSH）[11]）通过哈希函数将向量映射到哈希表的不同桶中，使得相似向量通常位于同一桶内。

基于图的方法通过构建一个邻近图，将向量检索转化为图的遍历问题。这类方法在索引构建阶段，将数据集中的每个向量表示为图中的一个节点，并根据向量间的距离或相似性建立边的连接。不同的图索引结构主要体现在其独特的赋边策略上。索引构建的核心思想源于小世界网络模型，旨在创建一个结构，使得从任意入口点出发，能在较少步数内到达查询点的最近邻。这种结构允许在搜索时使用贪婪算法，逐步逼近目标。然而，图索引设计面临稀疏性和稠密性的权衡：较稀疏的图结构每步计算代价低，而较稠密的图则可能缩短搜索路径。基于图的代表性方法有 NSW[32]、IPNSW[37] 和 HNSW[31] 等。

基于乘积量化的方法通过将高维向量空间划分为多个子空间，并在每个子空间中进行聚类得到码本和码字，以此作为构建索引的基础 [18]。这类方法主要包括训练和查询两个阶段。在训练阶段，该方法学习如何将高维向量最优地量化为码字 ID 序列。通常这个过程涉及将原始空间划分为多个子空间，在每个子空间内进行聚类，并通过聚类中心得到码字和码本。每个子空间内的聚类中心点即为码字，所有码字的集合构成码本。每个训练样本的每个子向量可以都用相应子空间的码字来近似，这样就实现了码字 ID 序列来表示训练样本，达到了数据量化的目的。在查询阶段，系统同样将查询向量划分为子向量，并在每个子空间中找到最近的码

字，得到码字 ID 序列。随后，系统计算查询向量的每个子向量到所有对应子空间的码字的距离，形成距离表。最后，系统利用这个距离表和数据库中每个向量的码字 ID 序列，快速查找并累加各个子向量的对应距离，得到查询向量与数据库向量之间的近似距离。通过对这些距离进行排序，系统最终得到最近邻结果。该方法在减少内存占用和加快距离计算速度方面表现出色，但量化过程中会不可避免地会引入一些误差。在某些需要精度更高的应用场景中，我们可以在 PQ 的基础上进一步进行精确排序，以得到精确的最近邻结果。此外，还有一些乘积量化的优化算法以及和其他索引相结合的算法，如 OPQ[15]、IVFPQ[19] 等。

2. 常见软件库介绍

在前文中，我们已经介绍了向量数据库的核心技术——相似度索引算法。这些算法是实现高效向量检索的关键。接下来，我们将介绍几种常用于构建和管理向量数据库的软件库，它们支持上述的相似性索引算法，是实现高效向量检索的重要工具。

Faiss⁴，由 Meta AI Research 开发，是一个专门优化密集向量相似性搜索和聚类的库。Faiss 提供了多种索引算法选择，这些算法涵盖了基于空间划分、基于量化以及基于图的方法等。这些算法不仅能在 CPU 上运行，部分算法还支持 GPU 加速，从而满足不同应用场景下的性能需求。然而，Faiss 本身并不是一个完整的数据库系统，而是一个功能强大的工具库。它专注于提供高效的索引和搜索功能，但在数据存储、管理、分布式支持和安全性措施等方面，Faiss 的功能相对有限。相比之下，向量数据库是一种更全面的解决方案，它不仅包括相似度索引算法，还整合了数据存储、管理、分布式支持和安全性措施等多方面的功能。截至目前，市场上已有多款成熟的向量数据库，如表6.1所示，它们适用于各种更复杂的 RAG 应用场景。

⁴<https://github.com/facebookresearch/faiss>

表 6.1: 常见的向量数据库。

向量数据库	URL	GitHub Star
milvus	https://github.com/milvus-io/milvus	28.4K
typesense	https://github.com/typesense/typesense	19.0K
qdrant	https://github.com/qdrant/qdrant	18.9K
chroma	https://github.com/chroma-core/chroma	13.7K
weaviate	https://github.com/weaviate/weaviate	10.4K
pinecone	https://www.pinecone.io/	×

6.3.5 检索结果重排

检索器可能检索到与查询相关性不高的文档。这些文档如果直接输入给大语言模型，可能会引发生成质量的下降。为此，在将其输入给大语言模型之前，我们还需要对其进行进一步的精选。精选的主要途径是对检索到的文档进行重新排序，简称重排，然后从中选择出排序靠前的文档。重排方法主要分为两类：基于交叉编码的方法和基于上下文学习的方法。

1. 基于交叉编码的重排方法

基于交叉编码的重排方法利用**交叉编码器**（Cross-Encoders）来评估文档与查询之间的语义相关性。关于交叉编码的介绍见第6.3.3节。MiniLM-L⁵是应用最为广泛的基于交叉编码的重排开源模型之一。该模型通过减少层数和隐层单元数来降低参数数量，同时采用知识蒸馏技术从大型、高性能的语言模型中继承学习，以此来提高模型性能。此外，还有其他一些在线重排模型可通过 API 直接访问，例如 Cohere⁶。对于希望探索其他高性能的重排器的读者，可以参考 MTEB⁷ 排行榜，该榜单汇集了很多性能优秀的重排模型。

2. 基于上下文学习的重排方法

基于上下文学习的方法是指通过设计精巧的 Prompt，使用大语言模型来执行

⁵<https://huggingface.co/cross-encoder/ms-marco-MiniLM-L-6-v2>

⁶<https://cohere.com/rerank>

⁷<https://huggingface.co/spaces/mteb/leaderboard>